you will be balancing between quality, latency, and cost (the larger your model, the more semantic information will be created, whereas the smaller the model, the more likely that it will meet your specific case). Within a production system, embeddings should be normalised and batched together, and to further boost retrieval, an additional layer of multilingual embeddings can support cross-lingual retrieval within the same pipeline.

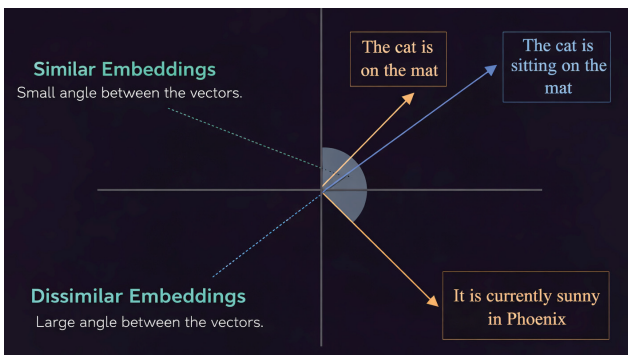

Figure 4: Embeddings

The embeddings are stored in a vector database that has been designed to operate using similarity search techniques. Because of this, there are several different index types available (some index types trade off speed for accuracy and others scale differently). Metadata filters also exist that can be used to filter through data based on the source, language, and section. Also, hybrid forms of retrieval exist that combine both the results from the similarity search and either keyword-based filtering or rule-based filtering. This will also provide a more precise and dependable answer in retrieval-augmented generators.

The way retrieval logic and context construction work

The query is embedded in a vector database for similarity-based retrieval so that there can be a large number (n = ?) of candidate chunks returned in order to maximise recall. Top-K tuning manages how many chunks are carried forward based upon the trade-offs between relevance, latency, and cost. To ensure there is no redundancy in the chunk candidates, a method called MMR Retrieval, which uses diversity to find relevant chunks, is often applied. Finally, chunks are reordered based on deeper levels of relevance through a re-ranking model (based on query–chunk or query–tag matches), so that the best chunks of context reach the LLM. Query rewriting may also be used to clarify intended meaning and to increase retrieval accuracy.

From the re-ranked results, only the top-K chunks are selected to fit within the LLM (limited tokens). The chunks will be ordered according to relevance and injected into

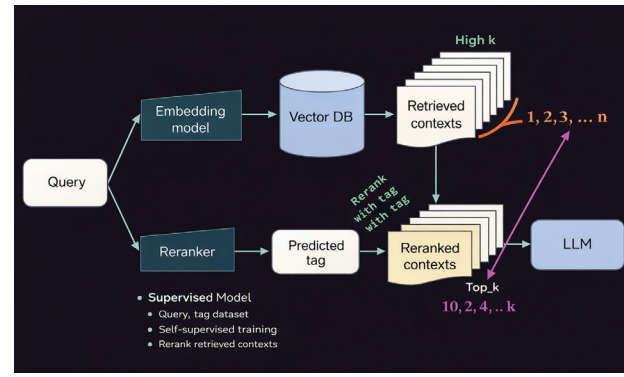

Figure 5: Retrieval logic and context construction

a structured prompt so that the instruction, context, and query are clearly separated. By controlling tight context and relevance filtering, we can minimise hallucinatory data so that responses are based on retrieved data.

Why evaluation and feedback loops are essential for production RAG

RAG evaluation assesses full cycle validation between queries, context retrieved and response generated. Retrieval metrics measure how well the retrieved context matches the query intent, i.e., do the pieces of content retrieved correspond to the intended query? The answer's quality is then measured by how well grounded the answer is, and whether it has relevance to the context from which it is being generated. These two sets of metrics ultimately contribute signals to help continuously improve query rewriting, retrieval ranking and selection of chunks.

Many pitfalls in production create direct breaks to the above-mentioned loop. For instance, over-chunking fragments the meaning and leads to reduced groundedness, whereas neglecting to use metadata results in irrelevant context returned. Sending too much context returned (retrieval inflation) wastes tokens and dilutes relevance, while failure to have fallbacks produces ungrounded answers when no high-quality context exists. To maintain RAG systems that produce reliable, hallucination-resistant outcomes, corrective measures must be taken.

You should now be able to understand the working of an end-to-end production-ready RAG system. Once you have this solid foundation, the building of RAG from the ground upwards will be methodical rather than overwhelming. Using these rules step by step will allow you to successfully transition from an experimental phase to an operationally sustainable, real-world RAG pipeline. END

By: Raj Patel
The author is a SaaS enthusiast and technical content creator.